

🔒 СВОДНЫЙ КОНСУЛИДИРОВАННЫЙ ПАСПОРТ БЕЗОПАСНОСТИ И АРХИТЕКТУРЫ «PG_RIGHTS_MANAGER»

Документ представляет собой техническое описание и архитектурный паспорт, объединяющий зоны ответственности, механизмы межмодульного взаимодействия и транзакционные барьеры всех пяти компонентов асинхронного веб-интерфейса разграничения доступа СУБД PostgreSQL.

--- Единая сквозная схема движения данных панели ---

ВХОДЯЩИЙ ПАКЕТ ОПЕРАТОРА (ФРОНТЕНД)

↓ Запрос к API периметру (main.py)

1. ВХОДНАЯ ПРОВЕРКА И ВАЛИДАЦИЯ JWT СЕССИИ

Роутинг main.py → Вызов Depends(RoleChecker(["admin", "Security_Manager", "auditor"]))

↓ Разветвление по исполнительным модулям

Модуль panel_users.py

ACID накат матрицы
внутренних лимитов
операторов

Модуль users.py

Веерный DDL накат ролей,
каскадная чистка ACL СУБД

Модуль smtp.py

Асинхронная почта,
TLS/STARTTLS доставка
токенов

↓ Сквозные инфраструктурные сервисы (utils.py)

- 🔒 **КРИПТОГРАФИЯ (Fernet)**: Расшифровка секретов серверов из RAM памяти процесса
- 🌀 **ПОТОКОБЕЗОПАСНЫЙ ПУЛ (Pyscopg3)**: Thread-Safe транзакции без Race Condition
- 📄 **СКВОЗНОЙ ИБ-АУДИТ (log_operation)**: Локализация try-except, инжект в audit_logs

↓ Фиксация транзакций кластера

**СЛУЖЕБНАЯ СУБД (service_manager_db) → УДАЛЕННЫЕ УЗЛЫ
POSTGRESQL**

--- Расширенный профиль информационной безопасности по файлам ---

1. Модуль служебного ядра и криптографии (utils.py)

- Уровень криптографической гигиены: Извлечение Fernet-ключей **MASTER_KEY** и параметров подключения строго из оперативной памяти (**os.getenv**), полностью исключая хранение секретов на накопителе хоста.
- Изоляция многопоточного рантайма: Паттерн Double-Checked Locking совместно с мьютексом **threading.Lock** предотвращает race condition и утечки дескрипторов сокетов.

- Отказоустойчивость подсистемы аудита: Функция `log_operation` изолирована верхнеуровневым затвором `try-except pass`. Переполнение диска логами или обслуживание базы логов не способно прервать или аварийно заблокировать основную бизнес-логику действий администратора.

2. Модуль веерного управления правами СУБД (users.py)

- Защита от инъекций динамического DDL: Все операции создания, модификации и блокировки ролей СУБД компилируются через токенизаторы `sql.Identifier` и `sql.Literal`.
- Ликвидация взаимных блокировок (Deadlocks): Команды очистки каталога `REASSIGN` и `DROP OWNED` разделены принудительной микропаузой ядра `time.sleep(0.1)`, предоставляя PostgreSQL достаточно времени для пересчета графа системных зависимостей в таблице `pg_shdepend`.
- Модель Single-Use Token: Одноразовые ссылки раскрытия паролей уничтожаются в БД панели командой `DELETE` внутри той же транзакции до момента вывода JSON на экран клиента.

3. Модуль автоматизации и оповещений (smtp.py)

- Предотвращение блокировок Event Loop: Тяжелые синхронные операции модификации файлов конфигурации `.env` вынесены из общего контекста в пул потоков через `run_in_threadpool`.
- Защита от MITM (Перехват сессии шлюза): Механизм автоматической сепарации портов (465 SSL / 587 STARTTLS) надежно блокирует проведение умышленных атак на понижение уровня безопасности (Downgrade Attacks).

4. Модуль администрирования панели (panel_users.py)

- ACID-транзакционность распределения лимитов: Сохранение допусков операторов к СУБД выполняется атомарно при `conn.autocommit = False`. При сбое транзакция полностью откатывается, исключая риски появления полунакатанных или битых лимитов в системе.
- Затвор самоликвидации ядра: Код эндпоинтов намертво блокирует любые операции деактивации, удаления или перевода в статус `NOLOGIN` для корневой системной учетной записи `admin`.

5. Главный служебный демон (main.py)

- Безопасность сетевой разведки (Инфраструктурные селекторы): Любые Ajax-запросы перечисления каталогов удаленных СУБД зажаты в жесткий лимит ожидания `connect_timeout=5` секунд. Сырые ошибки PostgreSQL перехватываются, логируются в ОС под тегом `[ИБ-КРАХ]` и маскируются в плоский JSON.